



17.2. The Sequence-to-Sequence Models

Learning to transform one sequence into another.

Previous Section:

 [17.1. The Transformer Models](#)

Contents:

[1. Introduction to Sequence-to-Sequence Models](#)

[1.1. Example: Machine Translation](#)

[1.2. Preparing Decoder Inputs](#)

[2. Types of Sequence-to-Sequence Models](#)

[2.1. Recurrent Neural Network Seq2Seq](#)

[2.2. Transformer Seq2Seq](#)

[2.3. Causal Mask in the Decoder](#)

[2.4. Inference with a Seq2Seq Transformer](#)

[3. Implementing a Simple Seq2Seq Model](#)

[Summary](#)

[References](#)

So far, most of the Natural Language Processing models we have built focused on **classification**. Given an input sentence, the model predicts a single label, such as positive or negative sentiment.

Many real-world NLP problems, however, require generating an entirely new sequence rather than predicting a single value.

For example, a machine translator receives an English sentence and produces a Spanish sentence. A summarization system receives a long article and generates a concise summary. Likewise, chatbots, question-answering systems, and text generation models all produce sequences of text as their output.

These problems belong to a family of models known as **Sequence-to-Sequence (Seq2Seq)** models.

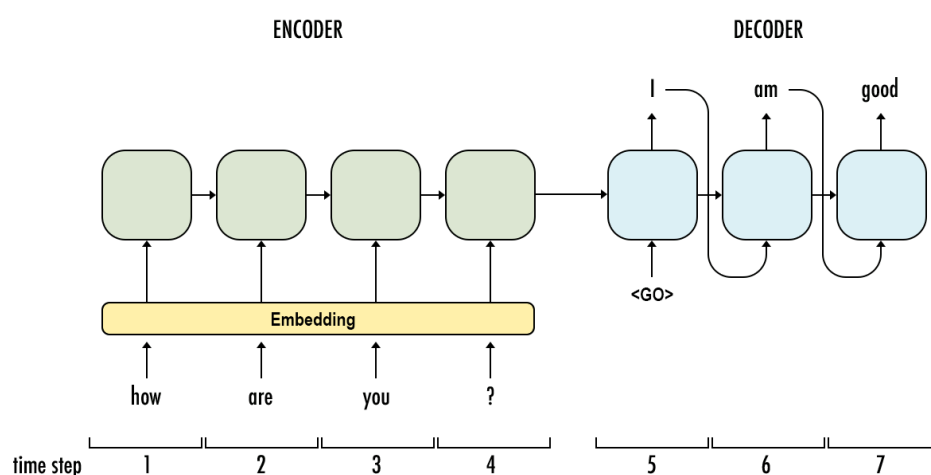
In this section, we explore how Seq2Seq models work, beginning with the classic RNN-based architecture before moving to the modern Transformer implementation that powers today's large language models.

1. Introduction to Sequence-to-Sequence Models

Unlike classification models, which map one input to one label, **Sequence-to-Sequence (Seq2Seq)** models map one sequence into another. Instead of producing a single prediction, the model generates an entire output sentence one token at a time.

This general framework is used in many Natural Language Processing tasks, including: Machine Translation; Text Summarization; Question Answering; Chatbots; Text Generation

Although these applications appear different, they all follow the same general architecture. A Seq2Seq model consists of two major components:



- An **Encoder**, which reads and understands the source sequence.
- A **Decoder**, which generates the target sequence.

The encoder first converts the input sentence into an internal representation. The decoder then uses this representation to generate the output sentence token by token until it produces a special **end token**, indicating that generation is complete.

1.1. Example: Machine Translation

One of the most common Seq2Seq applications is **machine translation**. Suppose we want to translate English sentences into Spanish.

Our dataset contains pairs of aligned sentences:

English Sentence		Spanish Sentence
How are you?	→	¿Cómo estás?
I like apples.	→	Me gustan las manzanas.
Good morning.	→	Buenos días.

Each row contains an English source sentence, and its corresponding Spanish translation.

Before training the model, we modify every Spanish sentence by adding two special tokens:

- **[start]** marks the beginning of the sentence.
- **[end]** marks the end of the sentence.

For example, `How are you? → [start] ¿Cómo estás? [end]`

These tokens allow the decoder to know when to begin and when to stop generating text.

Since English and Spanish have different vocabularies, we also build **two independent TextVectorization layers**: one for English, one for Spanish. Each language therefore has its own vocabulary and integer token mapping.

1.2. Preparing Decoder Inputs

Training the decoder requires a small but very important trick. Instead of feeding the complete target sentence directly into the decoder, we shift it by

one position.

Suppose the vectorized Spanish sentence is `[start] Me gustan las manzanas [end]`. We split it into two sequences.

- The decoder input becomes `decoder_inputs = spanish[:, :-1]` which removes the final token → `[start] Me gustan las manzanas`
- The expected output becomes `targets = spanish[:, 1:]` which removes the first token → `Me gustan las manzanas [end]`

Notice that the target sequence is shifted one position ahead. This means the decoder is always trained to predict the **next token**.

In other words, given `[start] Me gustan` → the model should predict `las`

At every time step, the decoder learns to predict token t_{i+1} from the encoded source sentence, and all previously generated target tokens $t_0, \dots, t_i$.

This training strategy is known as **teacher forcing**, because the correct previous token is always provided during training.

2. Types of Sequence-to-Sequence Models

Several architectures can implement the Seq2Seq framework. Historically, recurrent neural networks were introduced first, before being largely replaced by Transformers.

2.1. Recurrent Neural Network Seq2Seq

The original Seq2Seq architecture is built using two **Recurrent Neural Networks (RNNs)**.

- The **Encoder RNN** reads the source sentence one token at a time and gradually compresses the entire sentence into a hidden state. This hidden state serves as a summary of the input sentence.
- The **Decoder RNN** receives this hidden state as its initial state and begins generating the target sentence one token at a time until the complete sequence is produced.

A simple RNN can successfully model short sequences, but it struggles to retain information over long distances due to the **vanishing gradient problem**.

As the sequence becomes longer, important information from earlier tokens is gradually forgotten.

To alleviate this limitation, improved recurrent architectures such as **Long Short-Term Memory (LSTM)** and **Gated Recurrent Unit (GRU)** were introduced.

- **LSTM** introduces a memory cell together with input, forget, and output gates, allowing the network to preserve important information over much longer sequences.
- **GRU** simplifies the LSTM architecture by combining several gates into a single hidden state, making it computationally more efficient while often achieving similar performance.

Although LSTMs and GRUs significantly improve the ability of recurrent networks to capture long-range dependencies, they still process tokens sequentially.

This sequential nature leads to several remaining limitations:

- Tokens must be processed one after another, making parallel computation difficult.
- The encoder must compress the entire source sentence into a single hidden representation, which can lose information for very long sequences.
- During inference, an incorrect prediction early in the generated sentence may propagate errors to later tokens.

These limitations ultimately motivated the development of **Transformer-based Seq2Seq** models, which replace recurrence with attention mechanisms and allow much more efficient parallel processing.

2.2. Transformer Seq2Seq

Modern Seq2Seq models replace recurrent neural networks with Transformers.

- The **Transformer Encoder** first converts the source sentence into a sequence of rich, context-aware representations rather than compressing everything into a single vector.
- The **Transformer Decoder** then generates the translated sentence while attending to two different sources of information.

- First, it performs **self-attention** over the target tokens that have already been generated.
- Second, it performs **cross-attention**, allowing every generated token to directly access the encoded source sentence.

This additional attention mechanism enables the decoder to retrieve relevant information from any position in the source sequence instead of relying on a single compressed state.

As a result, Transformer-based Seq2Seq models generally produce more accurate translations than traditional RNN-based architectures.

2.3. Causal Mask in the Decoder

During training, the entire target sentence is already available. Without additional restrictions, the decoder could simply look at future tokens when predicting the next word.

For example, suppose we want to predict:

Me gustan las _____

If the decoder is allowed to see the future token **manzanas**, the task becomes trivial. The model would simply copy the correct answer instead of learning to generate language.

To prevent this, Transformer decoders use a **causal mask**. The mask blocks attention to every future position while allowing each token to attend only to itself and all previous tokens.

Conceptually,

$$M_{ij} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

where i is the current token being predicted; j is another token position; M_{ij} is the attention mask.

This means: If $j \leq i$, attention is allowed; If $j > i$, attention is blocked.

When the Softmax function is applied, entries with value $-\infty$ receive an attention weight of essentially zero, making future tokens invisible to the decoder.

2.4. Inference with a Seq2Seq Transformer

During inference, the decoder no longer has access to the correct target sentence. Instead, generation proceeds one token at a time.

- The process begins with the special token `[start]`.
- The decoder predicts the next token using the current decoded sentence together with the encoded source representation.
- The predicted token (typically selected using **argmax**) is appended to the decoded sentence and fed back into the decoder for the next prediction.
- This process repeats until either the decoder generates the `[end]` token, or the maximum allowed sequence length is reached.

By repeatedly generating one token at a time, the decoder gradually constructs the complete translated sentence from beginning to end.

3. Implementing a Simple Seq2Seq Model

Now that we understand how Sequence-to-Sequence models work, let us build a small toy example for **English-to-Spanish translation**.

The objective is not to build a production-quality translator, but rather to understand how Seq2Seq data is prepared before being fed into an encoder-decoder architecture.

We begin by creating a small collection of paired English and Spanish sentences.

```
# Some toy English-Spanish pairs for sequence-to-sequence.
pairs = [
    ("how is the weather today", "qué tiempo hace hoy"),
    ("i like this movie", "me gusta esta película"),
    ("she can play piano", "ella puede tocar piano"),
    ("this book is good", "este libro es bueno"),
```

```
]

source_texts = [src for src, tgt in pairs]
target_texts = [tgt for src, tgt in pairs]
```

Before training the decoder, we add two special tokens to every Spanish sentence.

- `[start]` indicates where decoding begins.
- `[end]` indicates where decoding should stop.

```
target_texts = ["[start] " + tgt + " [end]" for
                tgt in target_texts]
```

Since we want to preserve these special tokens, we customize the preprocessing step so that the square brackets remain while unnecessary punctuation is removed.

```
import string

strip_chars = string.punctuation + ";"
strip_chars = strip_chars.replace("[", "").replace("]", "")

def custom_standardization(input_string):
    lowercase = tf.strings.lower(input_string)

    return tf.strings.regex_replace(lowercase,
                                     f"[{re.escape(strip_chars)}]", "")
```

Next, we create two independent **TextVectorization** layers. Each language has its own vocabulary, so English and Spanish must be vectorized separately.

```
source_vectorization = TextVectorization(
    max_tokens=50,
    output_mode="int",
    output_sequence_length=8,
```

```

        standardize=custom_standardization
    )

    target_vectorization = TextVectorization(
        max_tokens=50,
        output_mode="int",
        output_sequence_length=9,
        standardize=custom_standardization,
    )

```

After creating the vectorizers, we build the vocabularies and convert every sentence into integer sequences.

```

source_vectorization.adapt(source_texts)
target_vectorization.adapt(target_texts)

source_encoded = source_vectorization(source_texts)
target_encoded = target_vectorization(target_texts)

```

As discussed earlier, the decoder is trained by predicting the **next token**. Therefore, we split the target sentence into two shifted sequences.

```

decoder_inputs = target_encoded[:, :-1]
target_outputs = target_encoded[:, 1:]

```

Here, `decoder_inputs` contains everything except the final token. And `target_outputs` contains everything except the first token.

The decoder therefore learns to predict each subsequent word given all previous words.

Creating a Causal Attention Mask

Transformer decoders must not look at future words during training. To enforce this constraint, we create a **causal attention mask**. The mask allows each position to attend only to itself and earlier positions.

```

def causal_attention_mask(size):
    mask = tf.linalg.band_part(tf.ones((size, size)), -1,

```

```
0)  
    return mask
```

This lower-triangular matrix contains **1** for positions that are visible, **0** for future positions that are masked.

Finally, we verify the generated tensors.

```
print("source_encoded shape:", source_encoded.shape)  
print("decoder_inputs shape:", decoder_inputs.shape)  
print("target_outputs shape:", target_outputs.shape)  
  
print(causal_attention_mask(8))
```

```
source_encoded shape: (4, 8)  
decoder_inputs shape: (4, 8)  
target_outputs shape: (4, 8)  
tf.Tensor(  
[[1. 0. 0. 0. 0. 0. 0. 0.]  
 [1. 1. 0. 0. 0. 0. 0. 0.]  
 [1. 1. 1. 0. 0. 0. 0. 0.]  
 [1. 1. 1. 1. 0. 0. 0. 0.]  
 [1. 1. 1. 1. 1. 0. 0. 0.]  
 [1. 1. 1. 1. 1. 1. 0. 0.]  
 [1. 1. 1. 1. 1. 1. 1. 0.]  
 [1. 1. 1. 1. 1. 1. 1. 1.]], shape=(8, 8), dtype=float32)
```

At this point, our toy dataset is fully prepared for training a Sequence-to-Sequence model. We have successfully: vectorized both languages, prepared shifted decoder inputs and targets, and generated the causal mask required by a Transformer decoder.

These are the essential preprocessing steps used by modern encoder-decoder architectures before the actual training process begins.

Summary

Sequence-to-Sequence (Seq2Seq) models learn to transform one sequence into another, making them suitable for tasks such as machine translation, text summarization, question answering, chatbots, and text generation.

In this section, we explored both **RNN-based** and **Transformer-based** Seq2Seq architectures, highlighting how an encoder processes the source sequence while a decoder generates the target sequence one token at a time. We also introduced **teacher forcing**, **cross-attention**, and **causal masking**, which enable Transformer decoders to generate text correctly without accessing future tokens.

Finally, we implemented a simple Seq2Seq preprocessing pipeline by vectorizing bilingual sentences, preparing decoder inputs and targets, and constructing a causal attention mask, completing the foundation needed for encoder-decoder models.

References

<https://www.geeksforgeeks.org/machine-learning/seq2seq-model-in-machine-learning/>

https://d2l.ai/chapter_recurrent-modern/seq2seq.html